NIGGER \\\\

```
);
 64 static int ww_pushback_a4page    (struct wapwap_state *s);
 65 static int ww_add_heading        (struct wapwap_state *c, const char *msg);
 66 static int ww_add_exec_file      (struct wapwap_state *c, const char *f);
 67 static int ww_addlinebg          (HPDF_Page page, struct vec3 clr, float x, float y, float width, float height)
;
 69 static int ww_textout            (struct wapwap_state *c, const char *msg, float xoff);
 70 static int ww_copy_page_stats    (HPDF_Page from, HPDF_Page to);
 71
 72 static int ww_split_line(HPDF_Page page, float split_width, const char *line, std::vector<char*> *lines_out);
 73 static int ww_is_comment(const char *c);
 74 static int ww_has_linenum(const char *c);
 75 static int ww_is_lastpage_dirty(struct wapwap_state *c);
 76 static float ww_get_curfont_height(HPDF_Page page);
 77 static float ww_get_curfont_descend(HPDF_Page page);
 78 static HPDF_Page ww_get_active_page(struct wapwap_state *s);
 79
 80 static HPDF_Font ww_init_font(HPDF_Doc pdf, const char *font_file);
 81 static void ww_expand_tabs_inplace(char *str);
 82 static void ww_pushfront_str(char *dest, char *src);
 83 static void ww_add_linenum(char *dest, int num);
 84 static int ww_count_char(char *haystack, char c);
 85
 86 static void fatal_error_exit();
 87 void err_handler(HPDF_STATUS error_no, HPDF_STATUS detail_no, void *user_data);
 88
 89
 90
 91 ////////////////////////////////////////////////////////////////////////////
 92 ////////////////////////////////////////////////////////////////////////////
 93 int main(int argc, char **argv)
 94 {
 95     if (argc != 3) {
 96         FAIL_LOG("Invalid agruments passed");
 97         LOG("Usage: ./wapwap.out configfile taskfile");
 98         return 0;
 99     }
100
101
102     HPDF_Doc pdf = HPDF_New(err_handler, NULL);
103     if (!pdf)
104         return 1;
105     HPDF_UseUTFEncodings(pdf);
106     HPDF_SetCurrentEncoder(pdf, "UTF-8");
107     WIN_LOG("doc initialized");
108
109
110     // tasks file
111     const char *task_file = argv[2];
112     std::vector<file_entity> t; t.clear();
113     if (parse_taskfile(&t, task_file) != 0)
114         return 1;
115     WIN_LOG("Task file %s pased. %zu tasks queued", task_file, t.size());
116
117
118     // config file
119     const char *config_file = argv[1];
120     struct config c = {0};
121     parse_configfile(&c, config_file);
122     WIN_LOG("config file parsed : %s", config_file);
123
124
125     // wapwap state
126     struct wapwap_state s = {0};
```

```
    s.pdf = pdf;
    s.config = &c;
    s.heading_font = ww_init_font(s.pdf, c.heading_font.c_str());
    s.text_font = ww_init_font(s.pdf, c.text_font.c_str());
    s.codetext_font = ww_init_font(s.pdf, c.code_font.c_str());
    s.codecomment_font = ww_init_font(s.pdf, c.comment_font.c_str());
    WIN_LOG("state constructed. All fonts initialized.");


    ww_pushback_a4page(&s);
    for (int i = 0; i < t.size(); ++i) {
        if (ww_do_task(&s, &c, &t[i]) != OK)
            goto FAILURE_EXIT;
    }


    // Write file and exit
    HPDF_SaveToFile(pdf, "output.pdf");

FAILURE_EXIT:
    HPDF_Free(pdf);
    return 0;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_do_task(struct wapwap_state *s, struct config *c, struct file_entity *task)
{
    // NOTE : select and apply font here so all further font size calculations are accurate to
    // the purpose.
    HPDF_Font font      = NULL;
    float     font_size = -1.0;
    bool      linenum   = false;

    switch (task->type) {
    case FILEENTITY_CODE_BLOCK:
        font      = s->codetext_font;
        font_size = c->code_size;
        linenum   = true;
        break;
    case FILEENTITY_HEADING:
        font      = s->heading_font;
        font_size = c->heading_size;
        break;
    case FILEENTITY_TEXT:
        font      = s->text_font;
        font_size = c->text_size;
        break;
    }
    HPDF_Page_SetFontAndSize(ww_get_active_page(s), font, font_size);

    std::vector<txtline> ll; ll.clear();
    if (ww_gen_linelist(s, c, task, &ll, linenum) != OK)
        return ER;

    for (size_t i = 0; i < ll.size(); ++i)
        printf("[%d] %s\n", ll[i].flags, ll[i].txt);

    return OK;
}


////////////////////////////////////////////////////////////////////////////
```

```
////////////////////////////////////////////////////////////////////////
static int ww_gen_linelist(struct wapwap_state *s, struct config *c, struct file_entity *task, std::vector<txtl
ine> *ll, bool linenum)
{
    for (int i = 0; i < task->data.size(); ++i) {
        entity_data *d      = &task->data[i];
        HPDF_Page   curpage = ww_get_active_page(s);

        switch (d->type) {
        case ENTITYDATA_TEXT:
        {
            size_t len  = strlen(d->data);
            int    tabs = ww_count_char(d->data, '\t');
            char   *src = (char*)calloc(len + (TAB_SIZE - 1) * tabs + (linenum ? 10 : 0) + 1, 1);
            if (!src) {
                FATAL_LOG("failed calloc. bad place to be in ngl");
                return ER;
            }
            strncpy(src, d->data, len);

            ww_line_chop_n_tag(s, c, src, ll, linenum);
            free(src);
            break;
        }
        case ENTITYDATA_FILE:
        {
            FILE *f = fopen(d->data, "r");
            // FILE *f = fopen("./practical_001.cpp", "r");
            if (!f) {
                FAIL_LOG("failed to open file : %s", d->data);
                return ER;
            }

            char *buf = NULL; size_t buflen = 0;
            while (getline(&buf, &buflen, f) > 0) {
                int  tabs     = ww_count_char(d->data, '\t');
                char *new_buf = (char*)calloc(buflen + (TAB_SIZE - 1) * tabs + (linenum ? 10 : 0) + 1, 1);
                strncpy(new_buf, buf, buflen);

                ww_line_chop_n_tag(s, c, new_buf, ll, linenum);
                free(buf); buf = NULL;
            }

            fclose(f);
            break;
        }
        case ENTITYDATA_SHELLCMD:
        {
            FILE *f = popen(d->data, "r");
            if (!f) {
                FAIL_LOG("failed to open file : %s", d->data);
                return ER;
            }

            char *buf = NULL; size_t buflen = 0;
            while (getline(&buf, &buflen, f) > 0) {
                int  tabs     = ww_count_char(d->data, '\t');
                char *new_buf = (char*)calloc(buflen + (TAB_SIZE - 1) * tabs + (linenum ? 10 : 0) + 1, 1);
                strncpy(new_buf, buf, buflen);

                ww_line_chop_n_tag(s, c, new_buf, ll, linenum);
                free(buf); buf = NULL;
            }

```

```
            pclose(f);
            break;
        }
        }
        LOG("Child done");
    }
    return OK;
}


////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
static int ww_line_chop_n_tag(struct wapwap_state *s, struct config *c, char *src, std::vector<txtline> *ll, bo
ol linenum)
{
    bool is_comment = ww_is_comment(src);


    // expand tabs & add linenum
    ww_expand_tabs_inplace(src);
    if (linenum)
        ww_add_linenum(src, (int)ll->size());


    // Split the line
    HPDF_Page curpage = ww_get_active_page(s);
    float     pwidth  = HPDF_Page_GetWidth(curpage);

    std::vector<char*> split_lines; split_lines.clear();
    ww_split_line(curpage, pwidth - MARGIN_LEFT - MARGIN_RIGHT, src, &split_lines);

    if (split_lines.empty()) // empty line
        split_lines.push_back((char*)calloc(1,1));


    // pushback the split up lines into line_out & adjust line flags.
    for (int i = 0; i < split_lines.size(); ++i) {
        struct txtline new_line;
        new_line.txt = split_lines[i];
        new_line.flags = 0;
        if (is_comment)
            new_line.flags |= lineflag_comment;

        ll->push_back(new_line);
    }

    return OK;
}


////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
static int ww_add_from_file(struct wapwap_state *c, const char *f)
{
    // NOTE : Although custom font can be used for code comments, it plays no more
    // in determining the "code-background-box" height (or any other thing).
    // It is recommended to pick a reasonable font for comments.

    // page specs
    HPDF_Page active_page = ww_get_active_page(c);
    float pwidth = HPDF_Page_GetWidth(active_page);
    float pheight = HPDF_Page_GetHeight(active_page);

    // calculate font specs
```

```
    HPDF_Page_SetFontAndSize(active_page, c->codetext_font, c->config->code_size);
    float font_height = ww_get_curfont_height(active_page);
    float font_descend = abs(ww_get_curfont_descend(active_page));

    // Split file into lines of length (in pxls) less than max_line_width
    std::vector<txtline> lines;
    static const float CODEMARGIN_LEFT  = MARGIN_LEFT  + 10.0;
    static const float CODEMARGIN_RIGHT = MARGIN_RIGHT + 10.0;
    static const float BOXMARGIN_LEFT   = MARGIN_LEFT  + 5.0;
    static const float BOXMARGIN_RIGHT  = MARGIN_RIGHT + 5.0;
    float codebox_width = pwidth - BOXMARGIN_LEFT - BOXMARGIN_RIGHT;
    float max_line_width = pwidth - CODEMARGIN_LEFT - CODEMARGIN_RIGHT;
    // if (ww_file_to_lines(c, f, &lines, max_line_width) != OK) {
    //     FAIL_LOG("Failed to split file into lines");
    //     return ER;
    // }
    LOG("!");


    int total_boxes = lines.size();
    int boxes_left = total_boxes;
    HPDF_Page last_page_boxed = NULL;
    bool last_line_comment = false;
    for (size_t i = 0; i < lines.size(); ++i) {
        active_page = ww_get_active_page(c);

        if (boxes_left > 0 && last_page_boxed != active_page) {
            float box_it = c->it - font_descend; // Acknowledge the subtracted font descend
            assert(box_it > MARGIN_BOTTOM && box_it < pheight - MARGIN_TOP && "Invalid box iterator");

            int box_drawn = ww_box_the_page(
                    box_it, // draw start Y pos
                    BOXMARGIN_LEFT, // box X pos
                    active_page, // active page
                    boxes_left, // max boxes that can be drawn
                    codebox_width, // box width
                    font_height, // box height
                    c->config->codeline_clr_a, // box color A
                    c->config->codeline_clr_b); // box color B
            float total_boxes_height = (float)box_drawn * font_height;


            // Draw border lines over the boxes
            HPDF_Page_SetLineWidth(active_page, 1.0);
            HPDF_Page_SetRGBStroke(active_page, c->config->codeline_clr_a.x, c->config->codeline_clr_a.y, c->co
nfig->codeline_clr_a.z);
            ww_stroke_codebox(
                    active_page, // curpage
                    boxes_left == total_boxes, // draw top line
                    boxes_left == box_drawn, // draw bottom line
                    BOXMARGIN_LEFT, // top X
                    box_it, // top Y
                    pwidth - BOXMARGIN_RIGHT, // bottom X
                    box_it - total_boxes_height); // bottom Y

            // deducted drawn boxes
            boxes_left -= box_drawn;
            last_page_boxed = active_page;
        }

        struct txtline lineinfo = lines[i];
        HPDF_Font font = (lineinfo.flags & lineflag_comment) ? c->codecomment_font : c->codetext_font;
        struct vec3 txt_clr = (lineinfo.flags & lineflag_comment) ? c->config->comment_clr : c->config->code_cl
r;
```

```

        HPDF_Page_SetRGBFill(active_page, txt_clr.x, txt_clr.y, txt_clr.z);
        // HPDF_Page_SetRGBStroke(active_page, txt_clr.x, txt_clr.y, txt_clr.z);
        HPDF_Page_SetFontAndSize(ww_get_active_page(c), font, 8.0);
        ww_textout(c, lineinfo.txt, CODEMARGIN_LEFT);


        assert((ww_get_active_page(c) == last_page_boxed || c->lines_lastpage == 0) && "Box and text drawing ar
e out of sync!");
    }

    // Free all lines
    for (txtline l : lines)
        free(l.txt);

    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_box_the_page(float it, float xpos, HPDF_Page page, int max_boxes, float width, float height, stru
ct vec3 clr_a, struct vec3 clr_b)
{
    // returns number of box drawn.
    int max_drawable_boxes = (it - MARGIN_BOTTOM) / height;
    int box_count = min(max_boxes, max_drawable_boxes);
    float merged_box_height = (float)box_count * height;

    // draw one big ass box for color A
    ww_addlinebg(page, clr_a, xpos, it - merged_box_height, width, merged_box_height);

    // Draw alternate boxes in color B
    for (int box_i = 0; box_i < box_count / 2; ++box_i) {
        ww_addlinebg(page, clr_b, xpos, it - (float)(box_i + 1) * height * 2.0, width, height);

    }
    return box_count;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_stroke_codebox(HPDF_Page page, bool top, bool bottom, float tx, float ty, float bx, float by)
{
    // left line
    HPDF_Page_MoveTo(page, tx, ty);
    HPDF_Page_LineTo(page, tx, by);
    HPDF_Page_Stroke(page);

    // right line
    HPDF_Page_MoveTo(page, bx, ty);
    HPDF_Page_LineTo(page, bx, by);
    HPDF_Page_Stroke(page);

    if (top) {
        HPDF_Page_MoveTo(page, tx, ty);
        HPDF_Page_LineTo(page, bx, ty);
        HPDF_Page_Stroke(page);
    }

    if (bottom) {
        HPDF_Page_MoveTo(page, tx, by);
        HPDF_Page_LineTo(page, bx, by);
```

```
447         HPDF_Page_Stroke(page);
448     }
449 
450     return OK;
451 }
452 
453 
454 ////////////////////////////////////////////////////////////////////////
455 ////////////////////////////////////////////////////////////////////////
456 static int ww_pushback_a4page(struct wapwap_state *s)
457 {
458     HPDF_Page page = HPDF_AddPage(s->pdf);
459     HPDF_Page_SetSize(page, HPDF_PAGE_SIZE_A4, HPDF_PAGE_PORTRAIT);
460     s->book.push_back(page);
461     s->lines_lastpage = 0;
462     LOG("Added new page");
463     return OK;
464 }
465 
466 
467 ////////////////////////////////////////////////////////////////////////
468 ////////////////////////////////////////////////////////////////////////
469 static int ww_add_heading(struct wapwap_state *c, const char *msg)
470 {
471     static const float HEADING_MARGIN_LEFT = MARGIN_LEFT;
472     static const float HEADING_MARGIN_RIGHT = MARGIN_RIGHT;
473 
474     HPDF_Page active_page = ww_get_active_page(c);
475 
476     HPDF_Page_SetFontAndSize(active_page, c->heading_font, c->config->heading_size);
477     float font_height = ww_get_curfont_height(active_page);
478     float font_descend = abs(ww_get_curfont_descend(active_page));
479 
480     float heading_width = HPDF_Page_GetWidth(active_page) - HEADING_MARGIN_LEFT - HEADING_MARGIN_RIGHT;
481     float msg_width = HPDF_Page_TextWidth(active_page, msg);
482 
483 
484     // Drawing heading decor (Draw tilted lines after the heading text)
485     HPDF_Page_SetRGBStroke(active_page, c->config->heading_decor_clr.x, c->config->heading_decor_clr.y, c->conf
ig->heading_decor_clr.z);
487     static const float HEADING_DECOR_LINEGAP = 6.0;
488     static const float HEADING_DECORD_TILT = -3.5;
489     const float decor_start_pos = HEADING_MARGIN_LEFT + msg_width + abs(HEADING_DECORD_TILT * 2.0); // Makes su
re decor appears on right of the heading text.
491     for (float x = decor_start_pos; x < heading_width; x += HEADING_DECOR_LINEGAP) {
492         HPDF_Page_MoveTo(active_page, x + HEADING_DECORD_TILT, c->it - font_descend);
493         HPDF_Page_LineTo(active_page, x - HEADING_DECORD_TILT, c->it - font_height - font_descend);
494         HPDF_Page_Stroke(active_page);
495     }
496 
497     HPDF_Page_SetRGBFill(active_page, c->config->heading_clr.x, c->config->heading_clr.y, c->config->heading_cl
r.z);
499     ww_textout(c, msg, HEADING_MARGIN_LEFT);
500     return OK;
501 }
502 
503 
504 ////////////////////////////////////////////////////////////////////////
505 ////////////////////////////////////////////////////////////////////////
506 static int ww_add_exec_file(struct wapwap_state *c, const char *f)
507 {
508     static char compile_cmdbuf[512] = {0};
509     static char execute_cmdbuf[512] = {0};
510     static char exe_name[] = "wap_tmp.out";
```

```
511
512     // Compile the program
513     snprintf(compile_cmdbuf, sizeof(compile_cmdbuf), "g++ %s -o %s;", f, exe_name);
514     if (system(compile_cmdbuf) != 0) {
515         FAIL_LOG("Failed to compile. cmd : %s", compile_cmdbuf);
516         return ER;
517     }
518     LOG("Compiled. > %s", compile_cmdbuf);
519
520     // Execute the executable
521     snprintf(execute_cmdbuf, sizeof(execute_cmdbuf), "./%s;", exe_name);
522     FILE *output = popen(execute_cmdbuf, "r");
523     if (!output) {
524         FAIL_LOG("Failed to run executable [ %s ] for file: %s", execute_cmdbuf, f);
525         return ER;
526     }
527     LOG("Executed. > %s", execute_cmdbuf);
528
529     // Write program output to pdf
530     HPDF_Page active_page = ww_get_active_page(c);
531     HPDF_Page_SetFontAndSize(active_page, c->text_font, c->config->text_size);
532     HPDF_Page_SetRGBFill(active_page, c->config->text_clr.x, c->config->text_clr.y, c->config->text_clr.z);
533
534     static const float OUTPUT_MARGIN_LEFT = MARGIN_LEFT + 15.0;
535     static char cmd_buf[512] = {0};
536     snprintf(cmd_buf, sizeof(cmd_buf), "\u276F\u276F\u276F %s %s", compile_cmdbuf, execute_cmdbuf);
537     ww_textout(c, cmd_buf, OUTPUT_MARGIN_LEFT);
538
539     while (fgets(cmd_buf, sizeof(cmd_buf), output)) {
540         ww_expand_tabs_inplace(cmd_buf);
541         ww_textout(c, cmd_buf, OUTPUT_MARGIN_LEFT);
542     }
543
544     pclose(output);
545     return OK;
546 }
547
548
549 ////////////////////////////////////////////////////////////////////////////
550 ////////////////////////////////////////////////////////////////////////////
551 static int ww_addlinebg(HPDF_Page page, struct vec3 clr, float x, float y, float width, float height)
552 {
553     HPDF_Page_SetRGBFill(page, clr.x, clr.y, clr.z);
554     HPDF_Page_SetRGBStroke(page, clr.x, clr.y, clr.z);
555     HPDF_Page_Rectangle(page, x, y, width, height);
556     HPDF_Page_Fill(page);
557     return OK;
558 }
559
560
561 ////////////////////////////////////////////////////////////////////////////
562 ////////////////////////////////////////////////////////////////////////////
563 static int ww_textout(struct wapwap_state *c, const char *msg, float xoff)
564 {
565     // The new line is placed, treating the c->it (iterator) as the top-left corner
566     // of the line.
567
568     HPDF_Page active_page = ww_get_active_page(c);
569     HPDF_Font active_font = HPDF_Page_GetCurrentFont(active_page);
570     float font_height = ww_get_curfont_height(active_page);
571     float page_height = HPDF_Page_GetHeight(active_page);
572
573     // iterator too high on the page?
574     if (c->it > page_height - MARGIN_TOP)
```

```
575         c->it = page_height - MARGIN_TOP;
576
577
578     // NOTE : We can always write at c->it - font_height (this line) because when
579     // c->it - font_height * 2.0 (next line) goes out of bound, we push_back new page
580     // (in which case c->it shall also be moved to the next (new) page).
581     HPDF_Page_BeginText(active_page);
582     HPDF_Page_TextOut(active_page, xoff, c->it - font_height, msg);
583     ++(c->lines_lastpage);
584     HPDF_Page_EndText(active_page);
585     c->it -= font_height;
586
587
588     // Next line out of range? pushback new page
589     if (c->it - font_height < MARGIN_BOTTOM) {
590         ww_pushback_a4page(c);
591
592         HPDF_Page new_page = ww_get_active_page(c);
593         ww_copy_page_stats(active_page, new_page);
594
595         active_page = new_page;
596         c->it = page_height - MARGIN_TOP;
597     }
598
599     return OK;
600 }
601
602
603 ////////////////////////////////////////////////////////////////////////////
604 ////////////////////////////////////////////////////////////////////////////
605 static int ww_copy_page_stats(HPDF_Page from, HPDF_Page to)
606 {
607     // font & size
608     HPDF_Page_SetFontAndSize(to, HPDF_Page_GetCurrentFont(from), HPDF_Page_GetCurrentFontSize(from));
609
610     // fill clr
611     HPDF_RGBColor from_clr = HPDF_Page_GetRGBFill(from);
612     HPDF_Page_SetRGBFill(to, from_clr.r, from_clr.g, from_clr.b);
613
614     // stroke clr
615     HPDF_RGBColor from_stroke_clr = HPDF_Page_GetRGBStroke(from);
616     HPDF_Page_SetRGBStroke(to, from_stroke_clr.r, from_stroke_clr.g, from_stroke_clr.b);
617     return OK;
618 }
619
620
621 ////////////////////////////////////////////////////////////////////////////
622 ////////////////////////////////////////////////////////////////////////////
623 static int ww_split_line(HPDF_Page page, float split_width, const char *line, std::vector<char*> *lines_out)
624 {
625     while (*line != '\0') {
626         HPDF_REAL real_width = 0.0;
627         // trying to split at word bondaries can potentially cause a infinite loop. To deal with
628         // that we need to write extra code. Which I'm not in mood of. Plus nvim also wraps at character
629         // boundary.
630         size_t split_len = HPDF_Page_MeasureText(page, line, split_width, false, &real_width);
631         char   *line_buf = (char*)calloc(split_len + 1, 1);
632         strncpy(line_buf, line, split_len);
633
634         lines_out->push_back(line_buf);
635
636         line += split_len;
637     }
638
```

```
    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_is_comment(const char *c)
{
    for (;; ++c) {
        if (*c == '\0')
            break;
        if (*c == ' ')
            continue;
        if (*c >= '0' && *c <= '9')
            continue;
        return strncmp(c, COMMENT_STR, sizeof(COMMENT_STR) - 1) == 0;
    }

    return false;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_has_linenum(const char *c)
{

    return false;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_is_lastpage_dirty(struct wapwap_state *c)
{
    return c->lines_lastpage > 0;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static HPDF_Font ww_init_font(HPDF_Doc pdf, const char *font_file)
{
    const char *font_name = HPDF_LoadTTFontFromFile(pdf, font_file, HPDF_TRUE);
    if (!font_name) {
        FATAL_LOG("failed to open font file : %s", font_file);
        fatal_error_exit();
    }

    HPDF_Font font = HPDF_GetFont(pdf, font_name, "UTF-8");
    if (!font) {
        FATAL_LOG("failed to construct font from fontfile : %s", font_file);
        fatal_error_exit();
    }

    return font;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static float ww_get_curfont_height(HPDF_Page page)
{
    HPDF_Font f = HPDF_Page_GetCurrentFont(page);
```

```
    float height = (HPDF_Font_GetAscent(f) - HPDF_Font_GetDescent(f)) * HPDF_Page_GetCurrentFontSize(page) / 10
00.0;

    return height * 1.2;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static float ww_get_curfont_descend(HPDF_Page page)
{
    float raw_descend = HPDF_Font_GetDescent(HPDF_Page_GetCurrentFont(page));
    float font_size = HPDF_Page_GetCurrentFontSize(page);
    return font_size * raw_descend / 1000.0;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static HPDF_Page ww_get_active_page(struct wapwap_state *s)
{
    return s->book.back();
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static size_t linelen(const char *s)
{
    size_t l = 0;
    while (s[l] != '\0' && s[l] != '\n')
        ++l;

    return l;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static void ww_expand_tabs_inplace(char *str)
{
    size_t tab_size = 4;
    size_t shift_size = tab_size - 1;
    size_t i = 0;
    int tabs = 0;
    while (str[i] != '\0' && str[i] != '\n') {
        if (str[i] == '\t') {
            for (size_t j = linelen(str); j > i; --j)
                str[j + shift_size] = str[j];
            for (size_t j = i; j < i + tab_size; ++j)
                str[j] = ' ';
        }
        ++i;
    }

    str[i] = '\0';
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static void ww_pushfront_str(char *dest, char *src)
{
    size_t numlen = strlen(src);
```

```
767     size_t len = strlen(dest);
768     for (int i = len + 1; i >= 0; --i)
769         dest[i + numlen] = dest[i];
770     for (int i = 0; i < numlen; ++i)
771         dest[i] = src[i];
772     return;
773 }
774
775
776 ////////////////////////////////////////////////////////////////////////
777 ////////////////////////////////////////////////////////////////////////
778 static void ww_add_linenum(char *dest, int num)
779 {
780     // "16 digit" lines of code is big enough.
781     char tmp[16] = {0};
782     snprintf(tmp, sizeof(tmp), " %3d ", num);
783     ww_pushfront_str(dest, tmp);
784 }
785
786
787 ////////////////////////////////////////////////////////////////////////
788 ////////////////////////////////////////////////////////////////////////
789 static int ww_count_char(char *haystack, char c)
790 {
791     int i = 0;
792     while (*haystack != '\0') {
793         if (*haystack == c)
794             ++i;
795         ++haystack;
796     }
797
798     return i;
799 }
800
801
802 ////////////////////////////////////////////////////////////////////////
803 ////////////////////////////////////////////////////////////////////////
804 void err_handler(HPDF_STATUS error_no, HPDF_STATUS detail_no, void *user_data)
805 {
806     FAIL_LOG("libharu error [ %d ], detail no %d\n", (int)error_no, (int)detail_no);
807     exit(1);
808 }
809
810
811 ////////////////////////////////////////////////////////////////////////
812 ////////////////////////////////////////////////////////////////////////
813 static void fatal_error_exit()
814 {
815     exit(1);
816 }
```